

# Sapyon — Take-Home Assignment

Stack: React, Python, PostgreSQL

---

## Overview

Build **AgencyDesk** — a multi-tenant client & project management platform purpose-built for agency-client work. One deployment serves many agencies. Each agency manages its own clients and projects, and invites clients into a portal to track progress on their own work — and nothing else.

This is scoped around data modeling and access-control correctness, not screen count. Assume you'll use an LLM to help write code — that's expected. What we're evaluating is whether **you** understand the schema and edge cases well enough to defend them in a follow-up interview.

## Product Vision

AgencyDesk's full feature set looks like the table below. You are not building all of it — the "Required" column tells you what's in scope for this assignment.

| Feature                            | In This Assignment      |
|------------------------------------|-------------------------|
| Project & task boards              | Required                |
| Client portal (guest access)       | Required                |
| Time tracking                      | Required                |
| File uploads & approval            | Required                |
| Dashboards & reporting             | Required                |
| Client intake forms                | Bonus                   |
| Automations & notifications        | Bonus                   |
| Timeline / Gantt view              | Discuss only — no build |
| Full CRM (leads, deals, contracts) | Not in scope            |

## Required Scope

### Boards & tasks

- Projects belong to a client. Tasks belong to a project: status, priority, assignee, due date.
- A task can be marked internal (agency-only) or client-visible.

### Client portal

- Clients log in separately from agency staff and see only their own projects and client-visible tasks.
- Clients can comment on tasks; they cannot create tasks or change status.

### Time tracking

- Agency users log time entries against a task (duration, note, date).
- A simple per-project hours summary is enough — no invoicing needed.

### File uploads & approval

- Files can be attached to a task (design files, docs, etc.) and carry the same internal / client-visible flag as tasks and comments.
- A client can mark an uploaded file approved or needs changes.

### Dashboards & reporting

- One simple dashboard per project: task counts by status and hours logged, scoped to what the viewer (agency vs client) is allowed to see.

### Multi-tenancy & roles

- Every agency is an isolated tenant. One agency must never see another agency's data, under any endpoint — this is the main thing we're testing.
- Three roles: `agency_admin` (full access), `agency_member` (assigned projects only), `client_user` (own client's client-visible content only).

## Edge Cases We Will Probe

Your schema and API should handle each of these explicitly — not by accident:

- **Cross-tenant access** — Tenant A can't read/write Tenant B's data, even by guessing an ID.
- **Internal content leaking to clients** — check every code path, not just the main list view (search, filters, and comments are easy to miss).
- **One person, two agencies** — the same email may be a client contact for two different agencies with different roles in each. Your login and identity model must support this cleanly.
- **Invite races** — resending an invite shouldn't duplicate it; accepting the same invite twice shouldn't create two accounts.
- **Removing a team member mid-task** — decide what happens to tasks assigned to an `agency_member` removed from a project, and implement it.

## Required Write-Up

A short DESIGN.md (half a page) covering: how your schema enforces tenant isolation, how a client is blocked from internal content, how your identity model supports one person across two agencies, and one edge case you're proud of handling well. We'll dig into this in the interview — vague answers are a bigger flag than an imperfect build.

## Deliverables

- Git repo: backend, frontend, migrations, seed data (2+ agencies, a client\_user, a mix of internal/client-visible tasks).
- DESIGN.md and a README.md with setup instructions (under 10 minutes to run).
- Tests or a script proving tenant isolation and client-visibility rules actually hold.
- **Required:** a 3–5 minute Loom (or similar) video walking through the app and your schema. Submissions without a video will not be reviewed.

## Evaluation Criteria

| Area                 | What we're looking for                                                             |
|----------------------|------------------------------------------------------------------------------------|
| Schema design        | Normalization, constraints, and whether isolation is enforced by the schema itself |
| Multi-tenancy & RBAC | No cross-tenant leaks; roles correctly scoped                                      |
| Edge cases           | The five cases above handled explicitly, with tests                                |
| Code quality         | Clear layering, readable, consistent                                               |
| Understanding        | DESIGN.md and interview answers show real understanding                            |